Homework 5:

SVM, PCA, Classification, and Neural Network
CPE 486/586

Instructor: Rahul Bhadani

Due: November 26, 2026, 11:59 PM
135oints

You are allowed to use a generative model-based AI tool for your assignment. However, you must submit an accompanying reflection report detailing how you used the AI tool, the specific query you made, and how it improved your understanding of the subject. You are also required to submit screenshots of your conversation with any large language model (LLM) or equivalent conversational AI, clearly showing the prompts and your login avatar. Some conversational AIs provide a way to share a conversation link, and such a link is desirable for authenticity. Failure to do so may result in actions taken in compliance with the plagiarism policy.

Additionally, you must include your thoughts on how you would approach the assignment if such a tool were not available. Failure to provide a reflection report for every assignment where an AI tool is used may result in a penalty, and subsequent actions will be taken in line with the plagiarism policy.

Submission instruction:

You may either complete your entire homework on Python Notebook and submit a Google Colab link or you may choose a combination of a pdf and Google Colab Notebook. However, you must provide **publicly accessible** Google Colab URL on Canvas.

For .pdf on Canvas, follow the format `CPE486586-LastFirst-HW-XX`. For example, if your name is Sam Wells, your file name should be `CPE486586-WellsSam-HW-XX.pdf`.

Your submission must contain your name and UAH Charger ID or your UAH email address. Please number your pages as well.

Please write down your unique keyword corresponding to your package and your Github account URL. Do not provide a link to your Google Colab notebook. Upload your Google colab notebook to your repo in a new folder called HWNotebooks. Provide your GitHub commit hash. Your submitted notebook must be fully executed

with all cells outputs. Please make sure there are no error in your submitted notebook.

Theory

1 Distance between Hyperplanes (5 points)

Consider two hyperplanes:

$$H_1 : \mathbf{w}^\top \mathbf{x} = 52$$

and

$$H_2 : \mathbf{w}^\top \mathbf{x} = 64$$

where w is unknown.
What is the distance between the two hyperplanes H_1 and H_2?

2 Overfitting in SVM (5 points)

What strategy (such as a change in SVM formulation) would you take to avoid overfitting in the case of a Support Vector Machine?

3 K-means (5 Points)

Given $\{(1,2),(2,1),(4,5),(5,4)\}$, perform one iteration of K-mens with initial centroids at $(1,1)$ and $(5,5)$.

4 Anomaly Detection in Clustering (5 Points)

How would you use clustering techniques to detect anomaly?

5 Single Neuron Forward Pass (10 Points)

Calculate the output of a single neuron with:

- Inputs: $x_1 = 0.5$, $x_2 = 0.8$, $x_3 = 0.2$

- Weights: $w_1 = 0.6$, $w_2 = -0.3$, $w_3 = 0.4$
- Bias: $b = 0.1$
- Activation: ReLU function $f(z) = \max(0, z)$

Calculate: $a = f\left(\sum_{i=1}^{3} w_i x_i + b\right)$

6 Neural Network by Hand (20 points)

Let's consider features of two training samples $\mathbf{x} = [\mathbf{x}^{(1)}, \mathbf{x}^{(2)}] = [(x_1^{(1)}, x_2^{(1)}), (x_1^{(2)}, x_2^{(2)})] = [(2, 4), (4, 5)]$ and the response variable as $\mathbf{y} = [y^{(1)}, y^{(2)}] = [-1, 1]$. Note that when we don't write superscript, i.e. x_1 or x_2, we are simply considering features of a training sample without specifying which training sample.

Consider a classification problem for which we will develop a neural network model. The loss function to be considered is

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) = \mathcal{L} = -\frac{1}{n}\sum_{i=1}^{n}\left(y^{(i)}\log(\hat{y}^{(i)}) + (1 - y^{(i)})\log(1 - \hat{y}^{(i)})\right)$$

Note that this is similar to the problem demonstrated in the class except we will calculate the loss by taking both training samples.

Here,

$$\frac{\partial \mathcal{L}}{\hat{y}^{(1)}} = -\frac{y^{(1)}}{\hat{y}^{(1)}} + \frac{1 - y^{(1)}}{1 - \hat{y}^{(1)}}$$

and

$$\frac{\partial \mathcal{L}}{\hat{y}^{(2)}} = -\frac{y^{(2)}}{\hat{y}^{(2)}} + \frac{1 - y^{(2)}}{1 - \hat{y}^{(2)}}$$

Individual training sample i will lead to the gradient with respect to a weight W as:

$$\nabla_W \mathcal{L}^{(i)} = \frac{\partial \mathcal{L}}{W^{(i)}} = \frac{\partial \mathcal{L}}{\hat{y}^{(i)}} \times \frac{\hat{y}^{(i)}}{\partial z^{(i)}} \times \frac{\partial z^{(i)}}{\partial W^{(i)}} \tag{1}$$

In Equation (1), superscript (i) denotes that gradient is calculated for individual samples. Then we calculate the average gradient as

$$\nabla_W \mathcal{L} = \frac{1}{n}\sum_{i}^{n}\frac{\partial \mathcal{L}}{W^{(i)}} \tag{2}$$

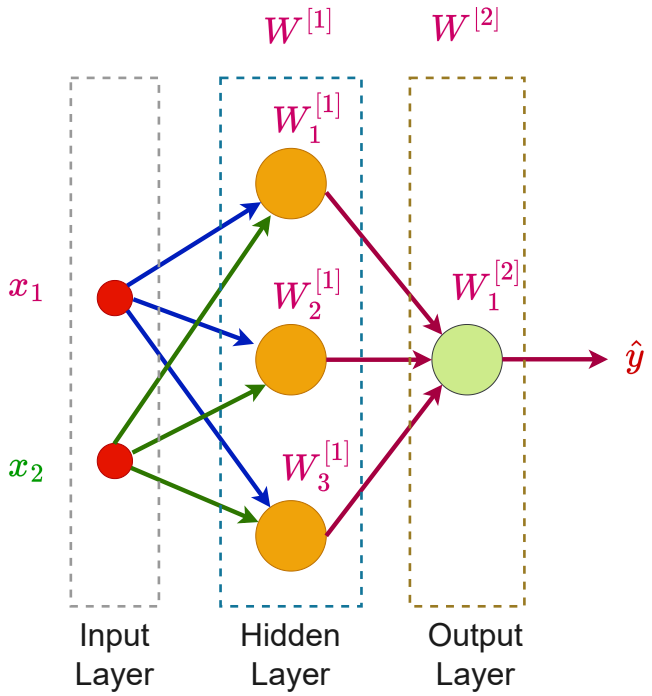


Figure 1: Homework 5:SVM, PCA, Classification, and Neural Network: Q6

so that the update rule for stochastic gradient descent becomes:

$$W = W - \eta \nabla_W \mathcal{L} \tag{3}$$

What it means is that you need to calculate the gradient for both training samples, take the average, and then update the weights.

Now, consider a neural network with one hidden layer containing three neurons so that the architecture looks as shown in Figure 1. In this problem, we won't consider bias, that is bias $b = 0$ at all neurons of the given Neural Network.

Further, let's consider the following initial values of weights:

$$\begin{aligned} W_1^{[1]} &= [0.1, 0.3] \\ W_2^{[1]} &= [0.2, 0.4] \\ W_3^{[1]} &= [0.1, 0.5] \\ W_1^{[2]} &= [0.1, 0.2, 0.1] \end{aligned} \tag{4}$$

Further, we will also consider the activation functions at all four neurons to be sigmoid, i.e.

$$a = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Note that the derivative of a with respect to z is given

$$\frac{\partial a}{\partial z} = \sigma(z)\Big(1 - \sigma(z)\Big)$$

You are required to complete the following task (only for one epoch) **(5 points each)**:
Use $\eta = 0.05$.

1. Perform the forward propagation on the first hidden layer and write down $z_1^{[1]}$, $z_2^{[1]}$, $z_3^{[1]}$, $a_1^{[1]}$, $a_2^{[1]}$, and $a_3^{[1]}$ for **both training samples**.
2. Perform the forward propagation on the output layer and write down $z_1^{[2]}$, $a_1^{[2]}$ for **both training samples**.
3. Calculate the final average loss value taking into account both training samples.
4. Perform the backward propagation to write down the updated values of $W_1^{[2]}$. Note the vector $W_1^{[2]}$ will contain three values as opposed to the two values you saw in the example done in the classroom. Don't forget to average the gradient using both training samples.

7 Dropout (5 points)

What is dropout in the context of a neural network and how is it beneficial in training a neural network?

Practice

8 Principal Component Analysis on Diabetes Dataset (20 points)

Download the diabetes dataset from the following link: https://github.com/rahulbhadani/Datasets/tree/master/Diabetes. Your task is to perform dimensionality reduction using Principal Component Analysis (PCA) and answer the following questions **(2.5 points each)**.

1. **Principal Component Analysis (PCA) with Two Components:** Perform PCA on the dataset, selecting only two principal components. Report the variance explained by these two components.

2. **2D Plot Analysis:** Generate a 2D plot using the two principal components obtained from the previous step. Analyze the plot and discuss whether the two classes (0 and 1 in the 'Outcome' column of the Diabetes dataset) are distinguishable.

3. **PCA with Three Components:** Now, perform PCA again, but this time select three principal components. Report the variance explained by each component and discuss the contribution of the additional third component.

4. **3D Plot Analysis:** Generate a 3D plot using the three principal components obtained from the previous step. Analyze the plot and discuss whether the two classes are more distinguishable in the 3D plot compared to the 2D plot.

9 Transforming Data with Kernels and SVM (20 points)

This assignment demonstrates how kernels allow SVMs to find complex, non-linear decision boundaries without explicitly transforming the data in the input space.

1. Generate a synthetic dataset that is not linearly separable using the following code (a classic "circles" dataset):

   ```
   from sklearn.datasets import make_circles
   X, y = make_circles(n_samples=400, noise=0.1, factor=0.3, random_state=1)
   ```

   Plot the data. Comment on why a linear SVM would fail here. **(3 points)**.

2. Implement a Linear SVM classifier and plot its decision boundary on the circles dataset. Confirm your hypothesis from the previous part by reporting the model's accuracy. **(4 points)**.

3. Now, implement a non-linear SVM using the Radial Basis Function (RBF) kernel with `gamma=1.0`. Plot the resulting decision boundary. Report the accuracy and describe how the kernel has solved the problem. **(5 points)**.

4. The RBF kernel has a `gamma` parameter. Investigate its effect by training models with `gamma=0.1` and `gamma=10`. Plot both decision boundaries. Explain the concepts of underfitting and overfitting in the context of the `gamma` parameter. **(8 points)**.

10 K-means (15 points)

Generate a half-moon dataset containing 500 samples using Python. Use `noise=0.2`. Implement a K-mean clustering on the half-moon dataset. Evaluate the cluster quality on four metrics: (i) Adjusted Rand Index, (ii) V-measure, (iii) Silhouette Score, and (iv) Calinski-Harabasz Index.

11 Classification with Neural Network (25 Points)

For this task, you will be working with the diabetes dataset (see https://github.com/rahulbhadani/Datasets/tree/master/Diabetes). Your goal is to implement a Neural Network model that can classify whether a given training sample indicates the presence of diabetes (Outcome = 1) or not (Outcome = 0).

You must provide a class based implement with the class name `TorchNet` with the following functionality scattered across multiple member function of class `TorchNet`:

(10 Points additional bonus if your neural network class can be supplied with input dimension and output dimension during object iniatiation.)

1. **Data Standardization (2 points)** Standardize the dataset using the StandardScaler from the sklearn library.
2. **Neural Network Implementation (5 points)** Implement a Neural Network with the following specifications:
   - The network should have two hidden layers.
   - The first hidden layer should contain 4 neurons.
   - The second hidden layer should contain 3 neurons.
3. **Training the Model (5 points)** Train the neural network using the Adam optimizer with a learning rate of 0.01. The training should be carried out for 1000 epochs. Choose the appropriate loss function for the classification problem.
4. **Loss Plot (3 points)** After training the model, plot the loss against the number of epochs.
5. **Model Saving (5 points)** Save the trained model for future use or deployment in ONNX format.
6. **Model Prediction (5 points)** Load the trained model you saved in the previous step, predict the outcome of diabetes (either 0 or 1) for the following data:

11.1 BONUS: (2 Points corresponding to each subpart above)

Add TorchNet class to the subpackage `model` in module neuralnet.py that you created previously. In your notebook, you must also show its uses by importing appropriate packages and subpackages. Please submit executed notebook with visible outputs

Feature	**Value**
Pregnancies	7
Glucose	149
BloodPressure	73
SkinThickness	94
Insulin	94
BMI	32
DiabetesPedigreeFunction	0.672
Age	45

Table 1: Dataset